

گزارش تحلیل و رمزشکنی رمز سزار

پروژه‌ی رمزنگاری کلاسیک

۱۷ مرداد ۱۴۰۵

فهرست مطالب

۱	مقدمه
۱	۲ مدل ریاضی
۲	۳ فضای کلید و پیچیدگی
۲	۴ پیاده‌سازی و تحلیل آماری
۲	۵ IC و آنتروپی
۲	۶ طراحی آزمایش
۲	۷ خروجی‌ها
۳	۸ نتیجه‌گیری

۱ مقدمه

رمز سزار یک رمز جانشینی ساده است که هر حرف انگلیسی را به اندازه‌ی یک کلید ثابت روی الفبا جابه‌جا می‌کند. این پروژه مدل ریاضی پیمانه‌ای، پیاده‌سازی، تحلیل آماری و آزمایش تجربی آن را بررسی می‌کند. فضای کلید فقط ۲۶ حالت دارد؛ بنابراین Force Brute برای سزار کاملاً عملی است و همین موضوع ضعف امنیتی آن را نشان می‌دهد.

۲ مدل ریاضی

حروف انگلیسی را با $A = 0$ تا $Z = 25$ نمایش می‌دهیم. برای x و کلید k داریم:

$$E_k(x) = (x + k) \bmod 26, \quad D_k(y) = (y - k) \bmod 26.$$

در نتیجه:

$$D_k(E_k(x)) = ((x + k) - k) \bmod 26 = x.$$

پس رمزگشایی معکوس رمزنگاری است. در پیاده‌سازی، case حفظ می‌شود و فاصله، عدد و علامت نگارشی تغییر نمی‌کنند.

۳ فضای کلید و پیچیدگی

فضای کلید $|\mathcal{K}| = 26$ است. برای متن به طول n ، بررسی تمام کلیدها هزینه $O(26n)$ دارد که از نظر مرتبه‌ی زمانی $O(n)$ است، چون ۲۶ ثابت است. این فضای کوچک باعث می‌شود مهاجم بتواند همه‌ی حالت‌ها را تقریباً آنی بررسی کند.

۴ پیاده‌سازی و تحلیل آماری

فایل `src/caesar.py` شامل `encrypt`، `decrypt` و `brute_force` است. فایل `src/analysis.py` فراوانی حروف، Chi-Square، IC n-gram و آنتروپی شانون را پیاده‌سازی می‌کند. روش اصلی تشخیص خودکار کمینه‌کردن رابطه‌ی زیر برای خروجی هر ۲۶ کلید است:

$$\chi^2 = \sum_{i=1}^{26} \frac{(O_i - E_i)^2}{E_i}.$$

۵ IC و آنتروپی

برای فراوانی‌های f_i و تعداد کل حروف N :

$$IC = \frac{\sum_i f_i(f_i - 1)}{N(N - 1)}.$$

چون سزار فقط برچسب حروف را جایگشت می‌کند، $IC(P) = IC(C)$ است و IC به تنهایی کلید را مشخص نمی‌کند. همچنین:

$$H(X) = - \sum_i p_i \log_2(p_i), \quad H(P) = H(C).$$

۶ طراحی آزمایش

برای طول‌های ۱۰، ۲۰، ۵۰، ۱۰۰، ۲۰۰، ۵۰۰ و ۱۰۰۰، در هر طول ۱۰۰ آزمایش انجام می‌شود. در هر آزمایش قطعه‌ای از متن انگلیسی، کلیدی تصادفی، رمزنگاری و سپس رمزشکنی بدون کلید انجام می‌شود. نرخ موفقیت از رابطه‌ی زیر به دست می‌آید:

$$SuccessRate = \frac{CorrectAttempts}{TotalAttempts} \times 100.$$

seed تصادفی در برنامه مشخص است تا نتایج قابل‌بازتولید باشند. زمان اجرا نیز با تابع زیر اندازه‌گیری می‌شود:

`time.perf_counter()`

۷ خروجی‌ها

نتایج در فایل‌های زیر ذخیره می‌شوند:

- data/output/success_rate.csv
- data/output/runtime.csv
- data/output/letter_frequency.csv
- data/output/bigram_frequency.csv
- data/output/trigram_frequency.csv
- data/output/statistics.csv
- data/output/entropy_results.csv
- data/output/cracking_results.csv

سه نمودار اصلی نرخ موفقیت، زمان اجرا و فراوانی حروف در پوشه‌ی docs/images قرار می‌گیرند.

۸ نتیجه‌گیری

Force Brute برای رمز سزار کاملاً عملی است، زیرا فضای کلید آن بسیار کوچک است. Chi-Square با افزایش طول متن معمولاً کلید را بهتر تشخیص می‌دهد، اما در متن کوتاه ابهام آماری ممکن است باعث خطا شود. IC و آنتروپی تغییر نمی‌کنند، زیرا سزار تعداد وقوع نمادها را عوض نمی‌کند. این پروژه نشان می‌دهد که حتی یک رمز ساده می‌تواند نمونه‌ی مناسبی برای آموزش حساب پیمانه‌ای، توابع معکوس، تحلیل آماری و طراحی آزمایش باشد.